

Trabajo Práctico 3 – Sistemas de Computación



Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1	Introducción	3
2	MBR mínimo	4
2.1	Construcción del MBR	4
2.2	Prueba de arranque	4
3	UEFI y Coreboot	6
3.1	UEFI	6
3.2	Uso de UEFI	6
3.3	Casos de bugs de UEFI	6
3.4	Converged Security and Management Engine (CSME) y Intel MEBx	7
3.5	Coreboot	7
4	Linker	9
4.1	Comparativa de objdump y hd	9
5	Depuración con GDB Dashboard	12
6	Modo protegido	15
7	Conclusión	16
8	Conclusion – Respuestas solicitadas	17

1 Introducción

En el presente trabajo se busca analizar y experimentar con distintas etapas del proceso de arranque de una computadora, poniendo el foco en cómo un programa muy pequeño puede ser cargado por el firmware y tomar control de la ejecución desde los primeros instantes del encendido. Para ello se parte de la construcción de imágenes booteables mínimas en Assembly, con el objetivo de comprender el formato del MBR, la función del BIOS y la forma en que el código es ubicado y ejecutado en memoria.

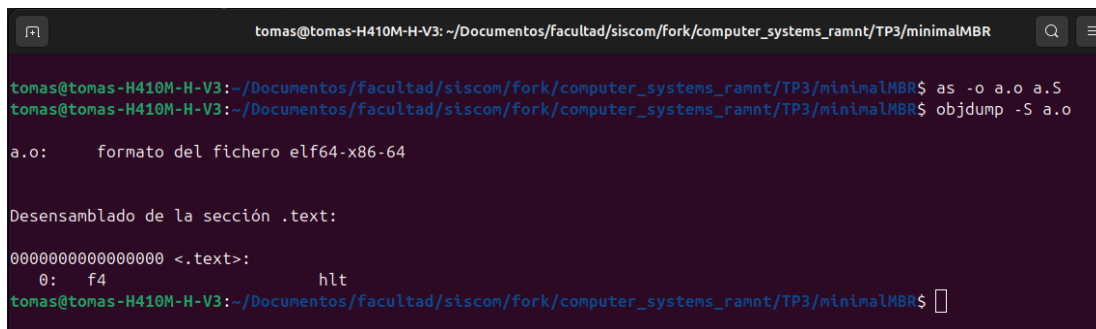
Además del desarrollo práctico, el informe pretende relacionar estas experiencias con conceptos más amplios del arranque moderno, incluyendo el rol de UEFI, coreboot y otros componentes del firmware. También se estudia la importancia del linker en la generación de imágenes binarias, el uso de herramientas como `objdump`, `hd` y GDB para inspeccionar y depurar el código, y finalmente la transición desde modo real hacia modo protegido mediante una GDT mínima. En conjunto, el propósito del trabajo es integrar teoría y práctica para comprender con mayor claridad cómo se organiza la secuencia de boot a bajo nivel.

2 MBR mínimo

En esta primera prueba se construyó un **MBR mínimo** cuyo único código de arranque es la instrucción `hlt`. El archivo `[a.S]` contiene solamente esa instrucción, por lo que el objetivo no fue mostrar funcionalidad visible, sino verificar el formato mínimo de una imagen booteable.

2.1 Construcción del MBR

Primero se ensambló el archivo y se comprobó con `objdump` que el código generado corresponde efectivamente a `hlt`.

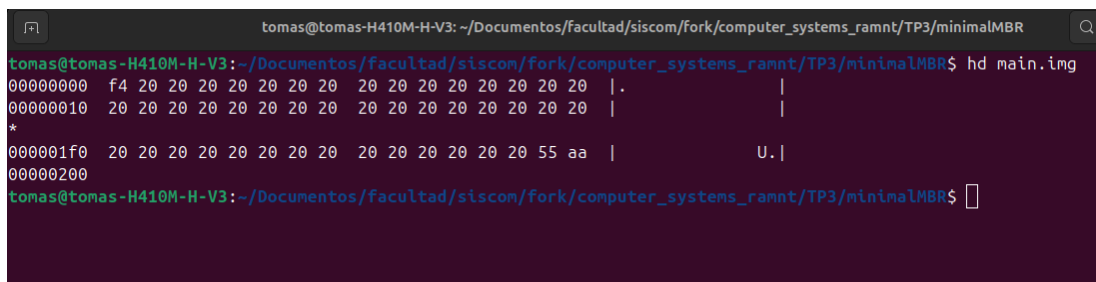


```
tomas@tomas-H410M-H-V3: ~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR$ as -o a.o a.S
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR$ objdump -S a.o

a.o:      formato del fichero elf64-x86-64

Desensamblado de la sección .text:
0000000000000000 <.text>:
   0:  f4                hlt
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR$
```

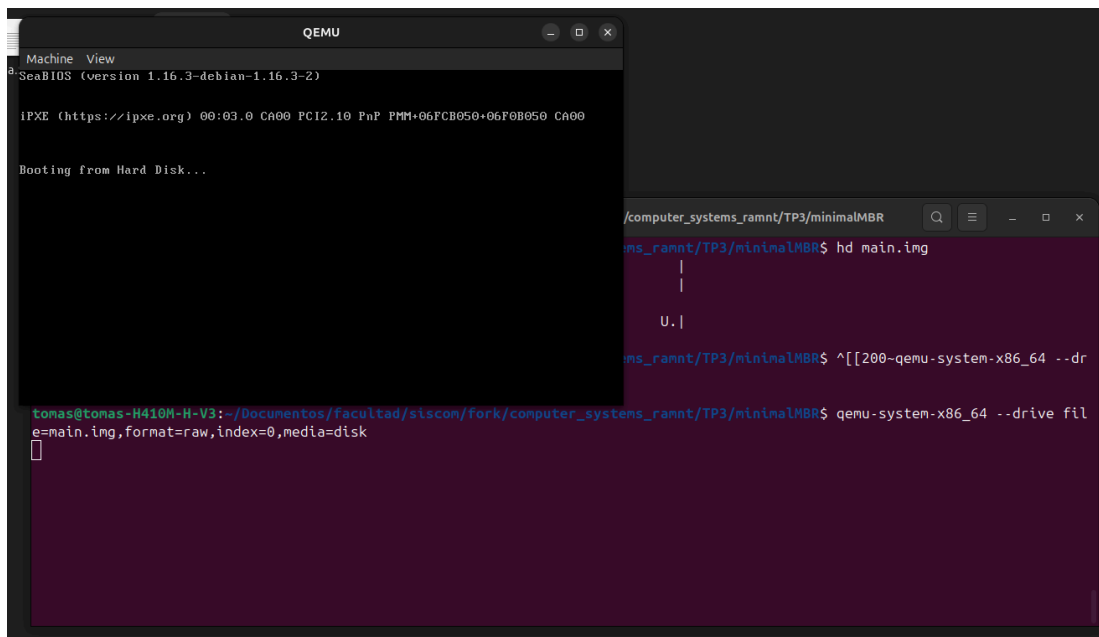
Luego se generó `main.img` con 512 bytes: el primer byte contiene `0xF4` (`hlt`) y al final del sector se colocó la firma de arranque `0x55AA`, que es la que reconoce el BIOS para considerar válido el MBR.



```
tomas@tomas-H410M-H-V3: ~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR$ hd main.img
00000000  f4 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |.      |
00000010  20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 |      |
*
000001f0  20 20 20 20 20 20 20 20 20 20 20 20 55 aa 00 00 |      U.|
00000200
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/TP3/minimalMBR$
```

2.2 Prueba de arranque

La imagen resultante se ejecutó con QEMU como disco booteable. El equipo virtual intenta arrancar desde el disco y, al ejecutar `hlt`, la secuencia se detiene inmediatamente, lo que confirma que el control llegó al código del MBR.



Como prueba adicional, la imagen también se copió a un pendrive con dd, lo que muestra que el archivo generado puede escribirse directamente como sector de arranque.

```
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscon/fork/computer_systems_ramnt/TP3/minimalMBR$ sudo dd if=main.img of=/dev/sdb
[sudo] contraseña para tomas:
1+0 records in
1+0 records out
512 bytes copied, 0,00537606 s, 95,2 kB/s
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscon/fork/computer_systems_ramnt/TP3/minimalMBR$
```

3 UEFI y Coreboot

3.1 UEFI

UEFI, sigla de **Unified Extensible Firmware Interface**, es una interfaz de firmware que reemplaza al BIOS tradicional en la mayoría de los equipos modernos. Su función es inicializar el hardware al encender la máquina, preparar el entorno mínimo de ejecución y transferir el control al cargador de arranque o al sistema operativo.

A diferencia del BIOS clásico, UEFI ofrece una arquitectura más modular, soporte para discos grandes mediante GPT, posibilidad de trabajar en 32 o 64 bits, controladores propios, variables persistentes en memoria no volátil y un modelo de servicios accesible mediante interfaces bien definidas.

3.2 Uso de UEFI

UEFI puede utilizarse de distintas maneras según el nivel de desarrollo que se quiera abordar. Desde la perspectiva de un usuario, se lo usa cuando el equipo arranca un sistema operativo en modo UEFI. Desde la perspectiva de un desarrollador de bajo nivel, puede interactuarse con UEFI escribiendo una aplicación EFI, es decir, un ejecutable con formato PE que el firmware carga y ejecuta antes de iniciar el sistema operativo.

Esa interacción se realiza a través de tablas y punteros a servicios que el firmware expone. Por ejemplo, una aplicación EFI puede invocar servicios de consola, acceso al sistema de archivos, manejo de memoria o consulta de variables del firmware. Una función que podría llamarse con esta dinámica es `OutputString`, perteneciente al protocolo de salida de texto simple, que permite escribir texto en pantalla durante la etapa de arranque. Esta función recibe una cadena en formato UTF-16 y la muestra por la consola de UEFI, por lo que resulta útil para programas de prueba, menús de arranque o mensajes de diagnóstico tempranos.

3.3 Casos de bugs de UEFI

Existen múltiples antecedentes de vulnerabilidades explotables en implementaciones UEFI. Este tipo de fallas es especialmente sensible porque se ejecuta antes del sistema operativo y con un alto nivel de privilegio.

Un caso conocido es **LogoFAIL**, una familia de vulnerabilidades publicadas en 2023 que afectó analizadores de imágenes usados por distintos fabricantes dentro del firmware UEFI. Mediante imágenes especialmente manipuladas, un atacante podía lograr ejecución de código durante el arranque y evadir mecanismos como Secure Boot.

Otro ejemplo importante es **BootHole**, divulgado en 2020. En ese caso, el problema estaba vinculado con GRUB2 en escenarios de arranque seguro y permitía, bajo ciertas condiciones, ejecutar código no confiable durante el proceso de boot. Aunque no todas estas fallas residen en el núcleo del estándar UEFI, sí impactan directamente en la cadena de arranque moderna basada en firmware UEFI.

3.4 Converged Security and Management Engine (CSME) y Intel MEBx

Converged Security and Management Engine (CSME) es un subsistema integrado en muchas plataformas Intel modernas. Funciona sobre un microcontrolador separado del procesador principal y permanece activo incluso cuando el sistema operativo todavía no arrancó o cuando la máquina está en ciertos estados de bajo consumo. Su propósito es proveer funciones de:

1. Seguridad
2. Gestión remota
3. Arranque confiable
4. Inicialización de plataforma
5. Servicios auxiliares para el firmware.

Dentro de ese ecosistema aparece **Intel Management Engine BIOS Extension (Intel MEBx)**, que es una interfaz de configuración accesible desde el firmware del equipo. MEBx permite ajustar parámetros relacionados con Intel Management Engine, especialmente en entornos corporativos donde se usan capacidades como administración remota, aprovisionamiento o políticas de seguridad.

3.5 Coreboot

Coreboot es un proyecto de firmware libre orientado a reemplazar BIOS o firmware propietario por una etapa de inicialización mínima, rápida y auditable. Su objetivo principal es realizar solo la configuración esencial del hardware y luego transferir el control

a una carga útil, llamada *payload*, como por ejemplo SeaBIOS, GRUB, LinuxBoot o incluso un kernel Linux.

En lugar de incorporar una gran cantidad de lógica cerrada dentro del firmware, coreboot reduce la complejidad visible y deja que componentes posteriores realicen tareas de más alto nivel.

Coreboot aparece en distintos tipos de productos. Uno de los casos más conocidos es el de varios **Chromebooks**, donde Google lo ha utilizado en numerosos dispositivos. También se encuentra en equipos de fabricantes o integradores enfocados en software libre y seguridad, como algunas configuraciones de **System76**, **Purism** y plataformas embebidas o de servidor específicas.

Además, coreboot es frecuente en entornos de laboratorio, investigación, proyectos educativos y hardware personalizado, donde resulta valioso contar con mayor control sobre la secuencia de arranque.

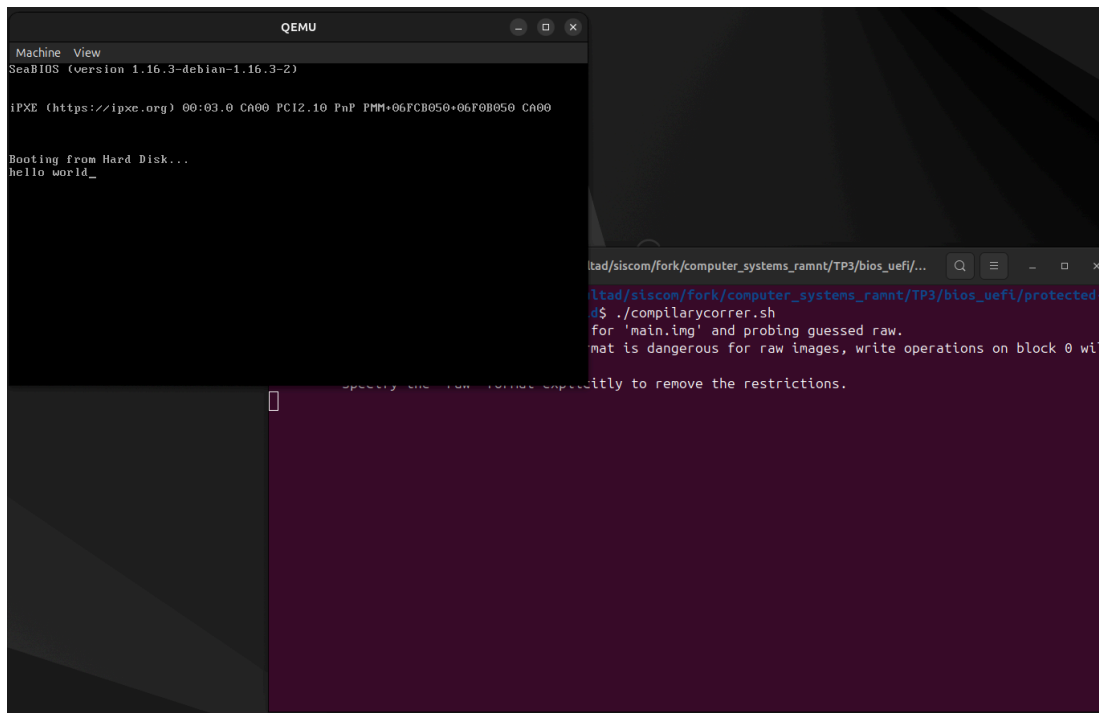
Las principales ventajas del uso de coreboot son las siguientes:

- Reduce el tiempo de arranque al minimizar la lógica ejecutada en firmware.
- Permite una mayor auditabilidad del código en comparación con soluciones completamente propietarias.
- Brinda mayor flexibilidad para elegir la carga útil más adecuada según el objetivo del sistema.
- Facilita escenarios de personalización, investigación y depuración del proceso de inicialización.
- Puede mejorar la seguridad al hacer más visible el comportamiento del firmware y reducir componentes innecesarios, aunque esto depende de la plataforma concreta.

4 Linker

4.1 Comparativa de objdump y hd

El archivo `compilarycorrerr.sh` automatiza el armado y la prueba del ejemplo. Primero ejecuta `as` para generar `main.o`, luego usa `ld` junto con `link.ld` para construir `main.img`, y finalmente corre esa imagen con QEMU. Gracias a eso, con un único script puede compilarse y verificarse el arranque del programa.



Al ejecutar el script, QEMU muestra en pantalla el mensaje `hello world`, lo que confirma que el sector fue cargado correctamente por el BIOS y que el código del bootloader llegó a ejecutarse.

Luego, al comparar las salidas de `objdump -S main.o` con `hd main.img`, se observa el mismo programa desde dos perspectivas distintas: `objdump` muestra el código ensamblado en forma de instrucciones, mientras que `hd` muestra la imagen final byte a byte.

```

tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems/protected-mode-sdc-master-01HelloWorld/01HelloWorld$ objdump -S main.o

main.o:      formato del fichero elf64-x86-64

Desensamblado de la sección .text:

0000000000000000 <loop-0x5>:
.code16
    mov $msg, %si
    0:  be 00 00 b4 0e          mov     $0xeb40000,%esi

0000000000000005 <loop>:
    mov $0x0e, %ah
loop:
    lodsb
    5:  ac                    lods    %ds:(%rsi),%al
    or %al, %al
    6:  08 c0                or      %al,%al
    jz halt
    8:  74 04                je      e <halt>
    int $0x10
    a:  cd 10                int     $0x10
    jmp loop
    c:  eb f7                jmp     5 <loop>

000000000000000e <halt>:
halt:
    hlt
    e:  f4                hlt

000000000000000f <msg>:
    f:  68 65 6c 6c 6f      push   $0x6f6c6c65
    14:  20 77 6f              and     %dh,0x6f(%rdi)
    17:  72 6c                jnb     85 <msg+0x76>
    19:  64                  fs

```

En la salida de `objdump` se ven desplazamientos pequeños dentro del archivo objeto, por ejemplo 0, 5, e y f. Eso ocurre porque `main.o` todavía es un objeto intermedio. La dirección real de carga `0x7c00` no aparece allí como dirección visible de ejecución, ya que `objdump` muestra offsets relativos dentro del objeto y no la ubicación final en memoria.

```

tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/
ected-mode-sdc-master-01HelloWorld/01HelloWorld$ hd main.img
00000000  be 0f 7c b4 0e ac 08 c0 74 04 cd 10 eb f7 f4 68 |..|.....t.....h|
00000010  65 6c 6c 6f 20 77 6f 72 6c 64 00 66 2e 0f 1f 84 |ello world.f...|
00000020  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
00000030  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000040  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
00000050  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000060  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84 00 |.f.....f....|
00000070  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
00000080  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000090  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
000000a0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000000b0  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84 00 |.f.....f....|
000000c0  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
000000d0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
000000e0  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
000000f0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000100  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84 00 |.f.....f....|
00000110  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
00000120  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000130  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
00000140  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
00000150  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84 00 |.f.....f....|
00000160  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
00000170  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
00000180  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
00000190  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000001a0  00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 1f 84 00 |.f.....f....|
000001b0  00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 66 66 |....f.....f|
000001c0  2e 0f 1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 |.....f.....|
000001d0  00 00 00 66 2e 0f 1f 84 00 00 00 00 66 2e 0f 0f |...f.....f..|
000001e0  1f 84 00 00 00 00 00 66 2e 0f 1f 84 00 00 00 00 |.....f.....|
000001f0  00 66 2e 0f 1f 84 00 00 00 00 0f 1f 00 55 aa |.f.....U.|
00000200

```

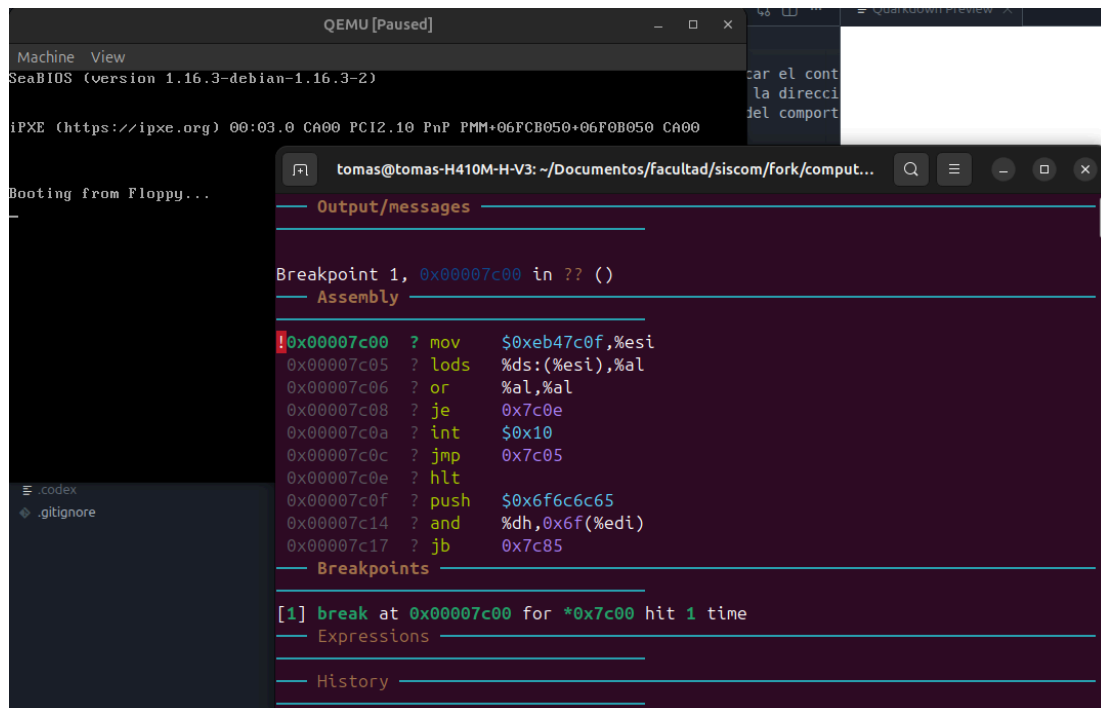
En `hd`, en cambio, puede verse la imagen final completa. Los primeros bytes del archivo corresponden al código máquina del programa, y enseguida aparece la cadena `hello world` en la columna ASCII. Esto indica que el programa fue colocado al comienzo del sector booteable. Sin embargo, tampoco aquí se ve de forma directa la dirección `0x7c00`, porque `hd` muestra el archivo en disco y no la memoria RAM. Lo que sí puede verificarse es que ese contenido será el que el BIOS copie luego a memoria en la dirección indicada por el linker.

También se observa al final de la imagen la firma `55 aa`, ubicada en el offset `0x1FE`. Esa es la representación en bytes de la firma de arranque definida en el linker mediante `SHORT(0xAA55)`. En otras palabras, en el script se expresa como `0xAA55`, pero en el volcado hexadecimal se ve almacenada como `55 aa` en los últimos dos bytes del sector.

Por lo tanto, `objdump` y `hd` permiten verificar el contenido del programa y su ubicación dentro de la imagen, mientras que la dirección de carga `0x7c00` se determina a partir del script del linker y del comportamiento estándar del BIOS.

5 Depuración con GDB Dashboard

Para depurar el programa se ejecutó QEMU con las opciones `-s -S`, lo que permite conectar GDB antes de que el bootloader comience a correr. Una vez hecha la conexión, GDB Dashboard muestra el estado de la CPU y el listado de instrucciones cargadas en memoria.



En esta captura puede verse de manera directa que la ejecución comienza en la dirección `0x7c00`. A diferencia de `objdump` y `hd`, aquí sí se observa la dirección real de memoria en tiempo de ejecución, ya que GDB está inspeccionando la máquina virtual ya cargada por el BIOS.

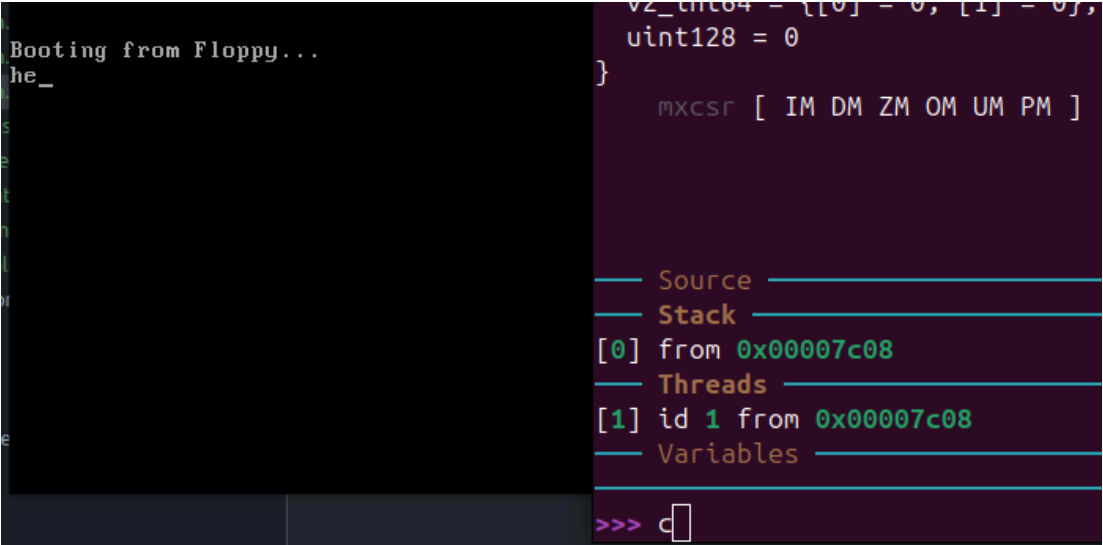
Luego se colocaron dos breakpoints manuales en las direcciones `0x7c08` y `0x7c0c`, que corresponden a las instrucciones `jz halt` y `jmp loop`. La idea fue detener la ejecución justo antes y justo después de la interrupción `int $0x10`, para observar cómo el programa recorre la cadena carácter por carácter.

```

>>> break *0x7c08
Punto de interrupción 2 at 0x7c08
>>>
break *0x7c0c
orden indefinida: «». Intente con «help»
>>> break *0x7c0c
Punto de interrupción 3 at 0x7c0c
>>> 

```

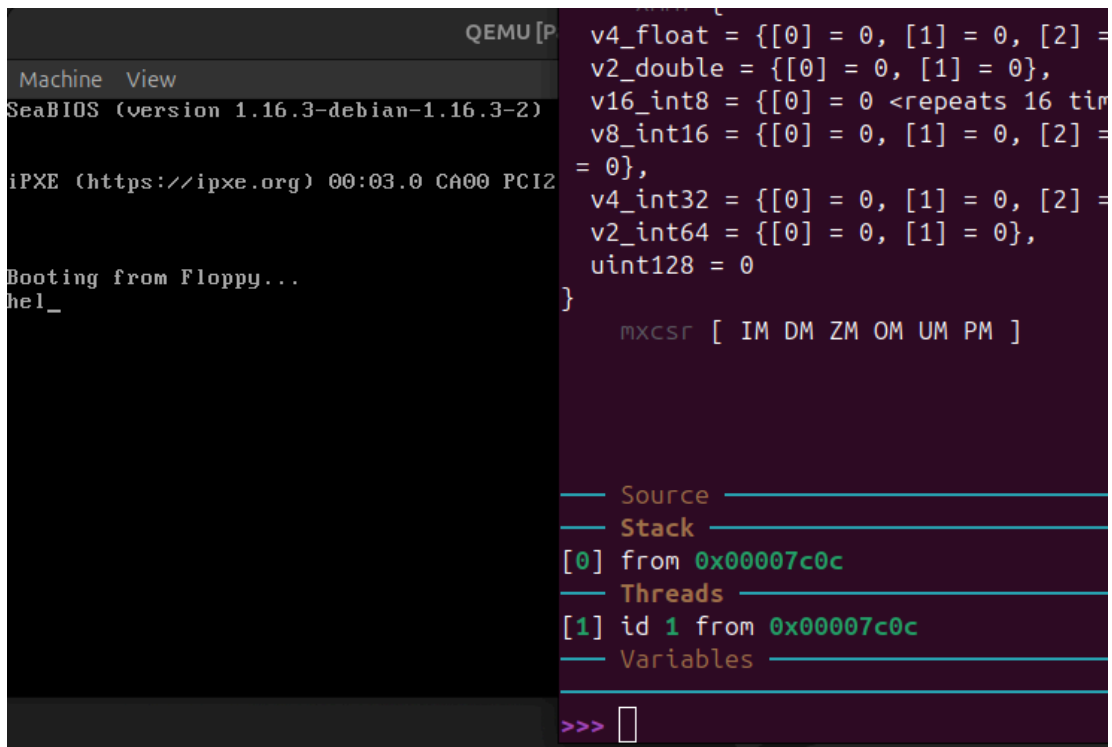
Al continuar la ejecución, el primer corte permite ver que ya se imprimieron los primeros caracteres de la cadena y que el flujo volvió al punto de decisión del bucle.



The screenshot shows a debugger window with two panes. The left pane displays the output of a program, which includes the text "Booting from Floppy..." and "he_". The right pane shows a stack trace with the following information:

- Source: [IM DM ZM OM UM PM]
- Stack: [0] from 0x00007c08
- Threads: [1] id 1 from 0x00007c08
- Variables: >>> c

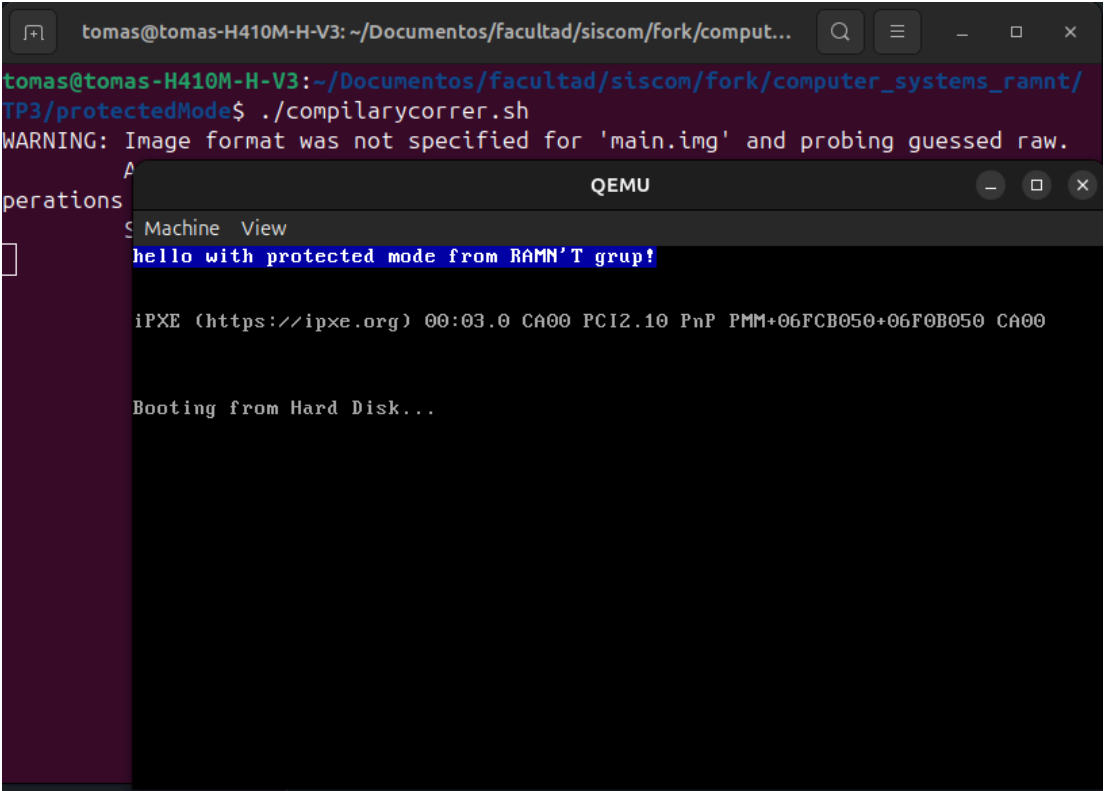
En la siguiente continuación se observa que la impresión avanza un carácter más. Esto confirma que en cada iteración se carga un byte de la cadena, se ejecuta la interrupción de video y luego se vuelve al inicio del bucle hasta encontrar el byte nulo final.



6 Modo protegido

En esta etapa se creó un nuevo archivo en Assembly para ejecutar nuevamente el programa de impresión, pero esta vez realizando antes la transición del procesador desde modo real a **modo protegido**. Para eso se definió una GDT mínima, se cargó con `lgdt`, se activó el bit PE de `CR0` y finalmente se hizo un salto largo al segmento de código de 32 bits.

Una vez en modo protegido, el programa ya no utiliza la interrupción BIOS `int 0x10` para mostrar texto, sino que escribe directamente en la memoria de video VGA en la dirección `0xb8000`. De ese modo se mantiene la funcionalidad de impresión, pero usando un mecanismo compatible con el nuevo modo de ejecución.



```
tomas@tomas-H410M-H-V3: ~/Documentos/facultad/siscom/fork/comput...
tomas@tomas-H410M-H-V3:~/Documentos/facultad/siscom/fork/computer_systems_ramnt/
TP3/protectedMode$ ./compilarycorrer.sh
WARNING: Image format was not specified for 'main.img' and probing guessed raw.
A
operations
$ Machine View
hello with protected mode from RAMN'T grup!
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050 CA00
Booting from Hard Disk...
```

La captura muestra que el programa logró entrar en modo protegido y escribir el mensaje en pantalla correctamente, cumpliendo el mismo objetivo que en la etapa anterior, pero ahora con segmentación y ejecución de 32 bits.

7 Conclusión

El desarrollo de este trabajo permitió recorrer de forma práctica distintas etapas del proceso de arranque de una computadora, desde la construcción de un MBR mínimo hasta la ejecución de código en modo protegido. A lo largo de las pruebas se verificó cómo intervienen el BIOS, el linker, el formato de la imagen booteable y la organización de memoria para que un programa muy pequeño pueda tomar el control de la máquina desde el encendido.

Además, la comparación entre mecanismos tradicionales y modernos de arranque, como BIOS, UEFI y coreboot, ayudó a ubicar cada experiencia dentro de un contexto más amplio. En conjunto, el trabajo sirvió para comprender mejor la relación entre hardware, firmware y software de bajo nivel, y para mostrar que conceptos como sector de arranque, segmentación, GDT y cambio de modo no son solo teóricos, sino elementos concretos que pueden observarse y validarse mediante herramientas reales.

8 Conclusion – Respuestas solicitadas

¿Qué es UEFI? ¿como puedo usarlo? Mencionar además una función a la que podría llamar usando esa dinámica.

UEFI es una interfaz de firmware moderna que reemplaza al BIOS tradicional y provee un entorno de ejecución previo al sistema operativo. Su función es inicializar el hardware, exponer servicios básicos y luego transferir el control a un cargador de arranque o a una aplicación EFI. A diferencia del modelo clásico del BIOS, UEFI trabaja con una arquitectura más modular, servicios bien definidos y tablas de punteros que permiten interactuar con el firmware de forma estructurada.

Puede usarse de dos maneras principales. Desde el punto de vista del usuario, se usa cada vez que el equipo arranca un sistema operativo en modo UEFI. Desde el punto de vista del desarrollo, puede utilizarse escribiendo una aplicación EFI en C u otro lenguaje compatible, compilándola al formato PE/COFF y ejecutándola directamente desde el firmware antes de que cargue el sistema operativo.

Esa interacción se realiza a través de las tablas y protocolos que el firmware expone. Por ejemplo, una aplicación EFI puede acceder a consola, sistema de archivos, variables persistentes o servicios de memoria. Una función concreta que puede invocarse con esta dinámica es `OutputString`, perteneciente al protocolo de salida de texto simple (`Simple Text Output Protocol`), utilizada para imprimir mensajes en pantalla durante la etapa de arranque.

¿Menciona casos de bugs de UEFI que puedan ser explotados?

Existen antecedentes importantes de vulnerabilidades explotables en implementaciones UEFI y en componentes de la cadena de arranque moderna. Un caso conocido es **LogoFAIL**, publicado en 2023, donde se encontraron fallas en analizadores de imágenes embebidos en distintos firmwares UEFI. A través de imágenes manipuladas especialmente, un atacante podía alcanzar ejecución de código durante el arranque incluso antes del sistema operativo.

¿Qué es Converged Security and Management Engine (CSME), the Intel Management Engine BIOS Extension (Intel MEBx).?

Converged Security and Management Engine (CSME) es un subsistema presente en muchas plataformas Intel modernas. Opera sobre un microcontrolador separado del procesador principal que corre un SO basado en MINIXS y sigue activo incluso cuando el sistema operativo todavía no arrancó o cuando la máquina se encuentra en ciertos estados de bajo consumo. El CSME tiene su propia porción de memoria no volátil (NVM) dentro del chip de la memoria Flash del BIOS. Su rol incluye tareas de seguridad, gestión remota, arranque confiable, inicialización de la plataforma y otros servicios auxiliares relacionados con el firmware.

Ejemplo: Intel Boot Guard: Antes de que se vea el logo del sistema operativo, el CSME verifica que el firmware de la placa base no haya sido modificado por un malware. Si las firmas digitales del BIOS/UEFI sea la del fabricante, si no coinciden, la PC simplemente no arranca para proteger al usuario. Una vez que el BIOS es declarado “seguro”, este toma el control y busca el cargador de arranque (GRUB). Aquí es donde entra el Secure Boot.

Dentro de ese ecosistema se encuentra **Intel Management Engine BIOS Extension (Intel MEBx)**, que es la interfaz de configuración accesible desde el propio firmware que le permite al usuario comunicarse con el CSME. Cuando se cambia una opción en la MEBx se envía esa instrucción a través de una interfaz llamada HECI (Host Embedded Controller Interface).

¿Qué es coreboot? ¿Qué productos lo incorporan? ¿Cuales son las ventajas de su utilización?

Coreboot es un proyecto de firmware libre diseñado para reemplazar BIOS o firmware propietario por una etapa de inicialización mínima, rápida y auditable. Su filosofía consiste en realizar únicamente la configuración esencial del hardware y luego transferir el control a una carga útil (payload), como por ejemplo SeaBIOS, GRUB, LinuxBoot o incluso un kernel Linux.

Se encuentra en distintos tipos de productos. Un caso muy conocido es el de varios **Chromebooks**, donde Google lo utilizó en numerosos equipos. También aparece en algunas configuraciones de fabricantes orientados a software libre y seguridad, como **System76** y **Purism**, además de plataformas embebidas, equipos de laboratorio y hardware personalizado.

Sus ventajas principales son:

- Reduce el tiempo de arranque al minimizar la lógica ejecutada en firmware.
- Permite una mayor auditabilidad del código respecto de soluciones cerradas.

- Brinda flexibilidad para elegir la carga útil más adecuada según el objetivo del sistema.
- Facilita tareas de investigación, depuración y personalización del proceso de arranque.
- Puede mejorar la seguridad al hacer más visible el comportamiento del firmware y reducir componentes innecesarios.

¿Que es un linker? ¿Que hace?

Un **linker** (o enlazador) es un programa que forma parte de la cadena de herramientas de compilación. Su función es combinar uno o más archivos objeto (archivos `.o` generados por el ensamblador o compilador) y bibliotecas en un único archivo ejecutable.

Durante el proceso de enlazado, el linker realiza varias tareas cruciales:

- **Resolución de símbolos:** identifica referencias simbólicas entre archivos objeto (por ejemplo, una función definida en un módulo que es llamada desde otro) y asigna direcciones de memoria concretas a cada símbolo.
- **Reubicación:** determina y resuelve la ubicación de memoria absoluta donde deben cargarse estos códigos.
- **Generación de la salida:** produce un archivo ejecutable en el formato especificado (ELF, PE/COFF, binario plano, etc.) que contiene el código máquina completo y listo para ejecutarse.

En el caso de un MBR o bootloader, el linker es fundamental porque debe conocer exactamente dónde residirá el código en memoria (típicamente `0x7c00` para un sector de arranque) y generar instrucciones de salto y referencias de memoria con esas direcciones precalculadas.

¿Que es la dirección que aparece en el script del linker? ¿Porqué es necesaria?

La dirección que aparece en el script del linker (en este trabajo, `0x7c00`) es la **dirección base de carga** del programa. Corresponde a la ubicación en memoria física o virtual donde el firmware (BIOS o UEFI) colocará el código ejecutable.

Esta dirección es necesaria porque:

- El linker debe calcular las direcciones de todas las instrucciones, etiquetas y saltos relativos. Sin conocer dónde comienza el código en memoria, no puede generar referencias correctas.
- En el caso del MBR, el BIOS siempre carga el sector de arranque en la dirección `0x7c00`. El script del linker debe fijar esta dirección para que cuando el código se ejecute, todas

las referencias internas sean válidas.

En el script `link.ld`, la línea 7: `. = 0x7c00;` establece explícitamente que el contador de ubicación comienza en `0x7c00`. Esto garantiza que todas las direcciones calculadas sean relativas a ese punto de partida.

¿Para que se utiliza la opción `--oformat binary` en el linker?

La opción `--oformat binary` en el linker genera una salida en **formato binario plano**, sin encabezados, tablas de símbolos, información de reubicación ni otros metadatos típicos de formatos ejecutables como ELF o PE/COFF.

Esta opción es esencial para crear un MBR porque:

- El BIOS no espera un archivo ejecutable estructurado, sino un **sector crudo de 512 bytes** que pueda copiar directamente a memoria en `0x7c00` y ejecutar sin procesamiento adicional.
- Un archivo con formato ELF o similar contendría encabezados que ocuparían espacio valioso y resultarían en una estructura no válida para el BIOS.
- Con `--oformat binary`, el resultado es solo el código máquina puro, seguido de los datos necesarios (como la firma `0xAA55` al final), sin ningún metadato innecesario.

En este trabajo, gracias a esta opción, el linker genera exactamente 512 bytes con el contenido apropiado, que puede ejecutarse directamente como sector de arranque.

¿Cómo sería un programa que tenga dos descriptores de memoria diferentes, uno para cada segmento (código y datos) en espacios de memoria diferenciados?

En este caso, la GDT(Global Descriptor Table) contendría al menos tres descriptores: uno nulo (requerido por el estándar x86), uno para el segmento de código y otro para el segmento de datos. Cada descriptor definiría una **base** y un **límite** distintos, apuntando a regiones separadas de memoria.

La ventaja de esta arquitectura es que el procesador puede **validar los accesos**: si un acceso de datos intenta traspasar el límite, o si se intenta ejecutar código desde un registro que no es el que le corresponde, se genera una excepción de protección general. Esto facilita la detección de errores y aumenta la robustez del sistema.

Cambiar los bits de acceso del segmento de datos para que sea de solo lectura, intentar escribir, ¿Que sucede? ¿Que debería suceder a continuación?

Cuando se modifica el byte de acceso del descriptor de datos para que no admita escritura (cambiando 0b10010010 por 0b10010000), el segmento queda configurado como **solo lectura**.

¿Qué sucede?

Si un programa intenta ejecutar una instrucción de escritura (como `mov [dirección], valor`) en ese segmento, el procesador detecta la violación de protección y genera una **excepción de protección general** (General Protection Fault, GPF). En un sistema con un manejador de excepciones completo, esta excepción podría capturarse y manejarse apropiadamente.

¿Qué debería suceder a continuación?

En un entorno de producción completo:

1. El procesador genera una **interrupción de excepción** (típicamente `int 0x0D` para GPF).
2. El sistema operativo o firmware consulta la IDT (Interrupt Descriptor Table) para encontrar el manejador de la excepción.
3. El manejador puede tomar distintas acciones: registrar el error, abortar el programa, generar un signal al proceso, etc.

Sin embargo, en este trabajo experimental con un programa mínimo que carece de una IDT completa:

- El procesador no encuentra un manejador válido para la excepción.
- La máquina virtual (QEMU) queda en un estado inconsistente y se reinicia repetidamente, tal como se observó al intentar la experiencia.

Esto demuestra que la protección de segmento es efectiva: detecta intentos ilícitos de acceso, pero también resalta la necesidad de una infraestructura de excepciones robusta para manejar tales situaciones de forma controlada.

En modo protegido, ¿Con qué valor se cargan los registros de segmento? ¿Porque?

En modo protegido, los registros de segmento (CS, DS, ES, FS, GS y SS) se cargan con **selectores**, no con direcciones directas como en modo real. Un selector es un valor de 16 bits que contiene:

- Un **índice** que apunta a una entrada específica en la GDT (Global Descriptor Table) o LDT (Local Descriptor Table).
- Un **bit de tabla** que indica si usar GDT o LDT.
- Un **nivel de privilegio** (RPL: Requested Privilege Level).

En este trabajo se utilizan:

- **Selector 0x08** para el segmento de código (CS). Este valor apunta al segundo descriptor de la GDT (la entrada nula ocupa el slot 0, por lo que el índice 1 resulta en $0x08 = 1 \ll 3$).
- **Selector 0x10** para el segmento de datos (DS). Este valor apunta al tercer descriptor de la GDT (índice 2, lo que resulta en $0x10 = 2 \ll 3$).

¿Por qué es así?

Porque en x86, cada entrada de la GDT ocupa exactamente 8 bytes. El procesador calcula la dirección de un descriptor multiplicando el índice (extraído del selector) por 8. Así:

- Índice 0: descriptor nulo, offset 0x00
- Índice 1: descriptor de código, offset 0x08
- Índice 2: descriptor de datos, offset 0x10